System Design: Interview Ready

LLD (Low-Level Design)

Design Patterns

Answer:

Reusable solutions to common design problems that improve flexibility and maintainability.

Example:

“In my backend, I used Strategy pattern to switch between different payment methods like UPI and Card.”

Singleton

Answer:

Ensures only one instance of a class exists across the application.

Example:

“I use Singleton for DB connection so all services share a single connection instance.”

Builder

Answer:

Used to construct complex objects step-by-step.

Example:

“I used Builder to create API response objects with optional fields like pagination, filters, metadata.”

Decorator

Answer:

Adds new behavior to an object dynamically without modifying its code.

Example:

“I used Decorator to add logging and authentication layers on API handlers.”

Observer

Answer:

Notifies multiple dependent objects when one object changes state.

Example:

“In a notification system, when an order is placed, multiple services like email and SMS get notified.”

Strategy

Answer:

Allows switching between different algorithms at runtime.

Example:

“I used Strategy to switch between different pricing algorithms based on user type.”

Factory

Answer:

Simplify object creation process.

Example:

“I used Factory to create different user types like Admin or Customer based on role.”

Abstract Factory

Answer:

Creates families of related objects without specifying exact classes aka factory of factories.

Example:

“I used Abstract Factory to create UI components for web vs mobile themes.”

SOLID

Answer:

5 principles to write maintainable and scalable code (SRP, OCP, LSP, ISP, DIP).

Example:

“I separated business logic into services to follow Single Responsibility Principle.”

OOPs

Answer:

Programming paradigm using objects and concepts like encapsulation, inheritance, polymorphism.

Example:

“I used encapsulation to hide internal DB logic inside repository classes.”

🔹 HLD (High-Level Design)

State vs Stateless

Answer:

Stateful servers store client data; stateless servers don’t and rely on external storage.

Example:

“I prefer stateless APIs so I can scale horizontally and store session in Redis.”

Server vs Serverless

Answer:

Server = you manage infra; Serverless = cloud manages scaling and execution.

Example:

“I used Cloud Run (serverless) for APIs to auto-scale without managing servers.”

Rate Limiter (Token Bucket)

Answer:

Controls request rate using tokens; requests are allowed only if tokens are available.

Example:

“I implemented token bucket to limit login attempts to prevent abuse.”

Load Balancer

Answer:

Distributes incoming traffic across multiple servers.

Example:

“I used round-robin load balancing to distribute API traffic across instances.”

CAP Theorem

Answer:

System can guarantee only 2 out of Consistency, Availability, Partition tolerance.

Example:

“In distributed DB, we sacrifice consistency for availability during network failures.”

CP vs AP

Answer:

CP = Consistency + Partition tolerance, AP = Availability + Partition tolerance.

Example:

“Banking systems prefer CP, while social media prefers AP for better availability.”

Latency

Answer:

Time taken to process a request.

Example:

“I reduced latency by adding Redis cache for frequently accessed data.”

Throughput

Answer:

Number of requests handled per second.

Example:

“I improved throughput by scaling horizontally and adding async processing.”

Caching (Write-through / Read-through)

Answer:

Write-through: write to cache + DB together.

Read-through: cache fetches data if not present.

Example:

“I used read-through caching to reduce DB load for user profile data.”

API: Idempotency

Answer:

Same request multiple times gives same result.

Example:

“I used idempotency keys in payment API to avoid duplicate transactions.”

API: Versioning

Answer:

Maintains backward compatibility when APIs change.

Example:

“I used /v1 and /v2 endpoints to support old and new clients.”

SQL vs NoSQL

Answer:

SQL = structured, relational; NoSQL = flexible, scalable.

Example:

“I used PostgreSQL for transactions and MongoDB for flexible user data.”

Queue (Basics)

Answer:

Used for async processing and decoupling systems.

Example:

“I used a queue to process email notifications in background instead of blocking API.”

Horizontal vs Vertical Scaling

Answer:

Horizontal = add more machines; Vertical = increase machine power.

Example:

“I scaled horizontally using multiple API instances behind a load balancer.”

How to Use This in Interview

Speak in this format:

> “Definition → Why → Example”

Example:

> “Rate limiting controls how many requests a user can make. I used token bucket algorithm to prevent abuse in login APIs by limiting requests per minute.”